

Data Structure

13 + 14. Trie & Cache

13 + 14주차: 트라이, 캐시

목차 INDEX

01 Trie

- Prefix
- End Marker
- Prefix Search
- 자동완성

02 LRU Cache

- Cache
- Hit / Miss
- Recency
- Hash Table
- 이중 연결 리스트

03 Range Query

- Prefix Sum
- Range Sum
- Query / Update
- Segment Tree
- Fenwick Tree

01 Trie

Trie

문자열 저장의 문제

apple

application

apply

apt

apart

appeal

위 문자열들을 효율적으로 저장하는 방법은?
(당연히 하나하나 직접 저장 말고)

Prefix

prefix ★ +

미국·영국 ['pri:fiks]

🔊 All 🇺🇸 🇬🇧 🇦🇺 🇮🇳

명사

1. 접두사(unhappy의 un-이나 preheat의 pre-처럼, 다른 단어 앞에 붙어 그 단어의 뜻을 달라지게 하는 글자(들)) (→affix, suffix)
2. (다른 것) 앞에 오는[붙는] 것(단어·글자·숫자 등)

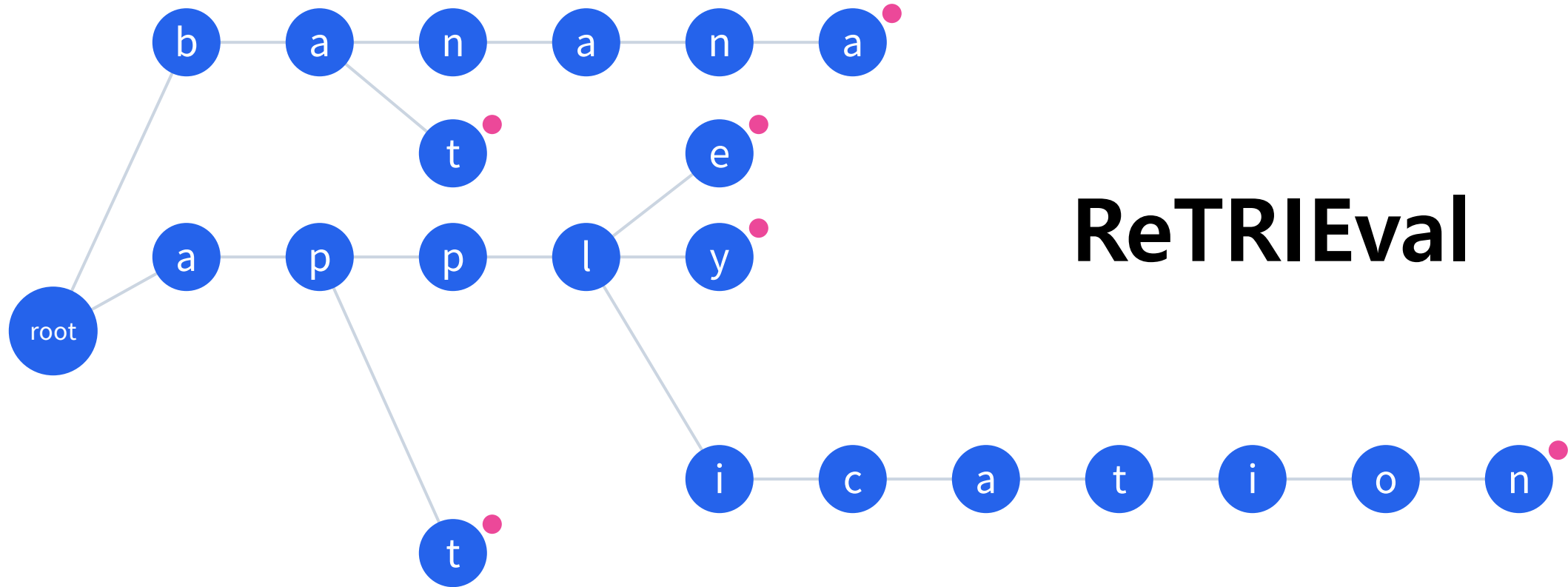
Car insurance policies have the prefix MC (for motor car). 🔊 🔄

자동차 보험 증권에는 앞에 (자동차를 나타내는) MC가 붙는다.

Prefix를 이용하면 문자열들을
효율적으로 저장할 수 있음

ap	p	le
ap	p	lication
ap	p	ly
ap	p	eal
ap		art
ap		t

Trie

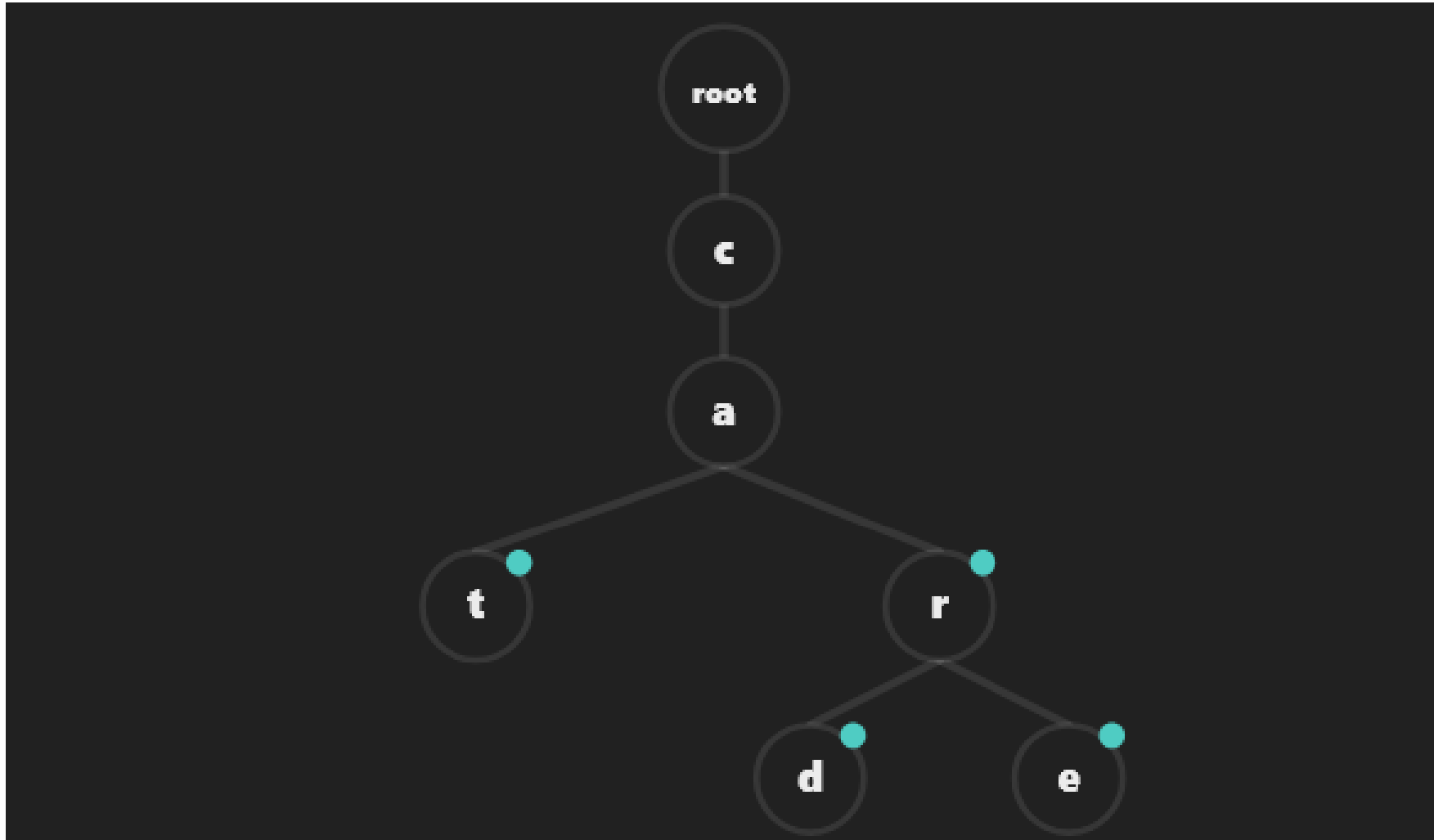


ReTRIEval

Trie는 트리 형태로 나타낼 수 있음
(분홍색 점은 단어의 끝을 표시)

01 Trie

Trie에 단어 삽입

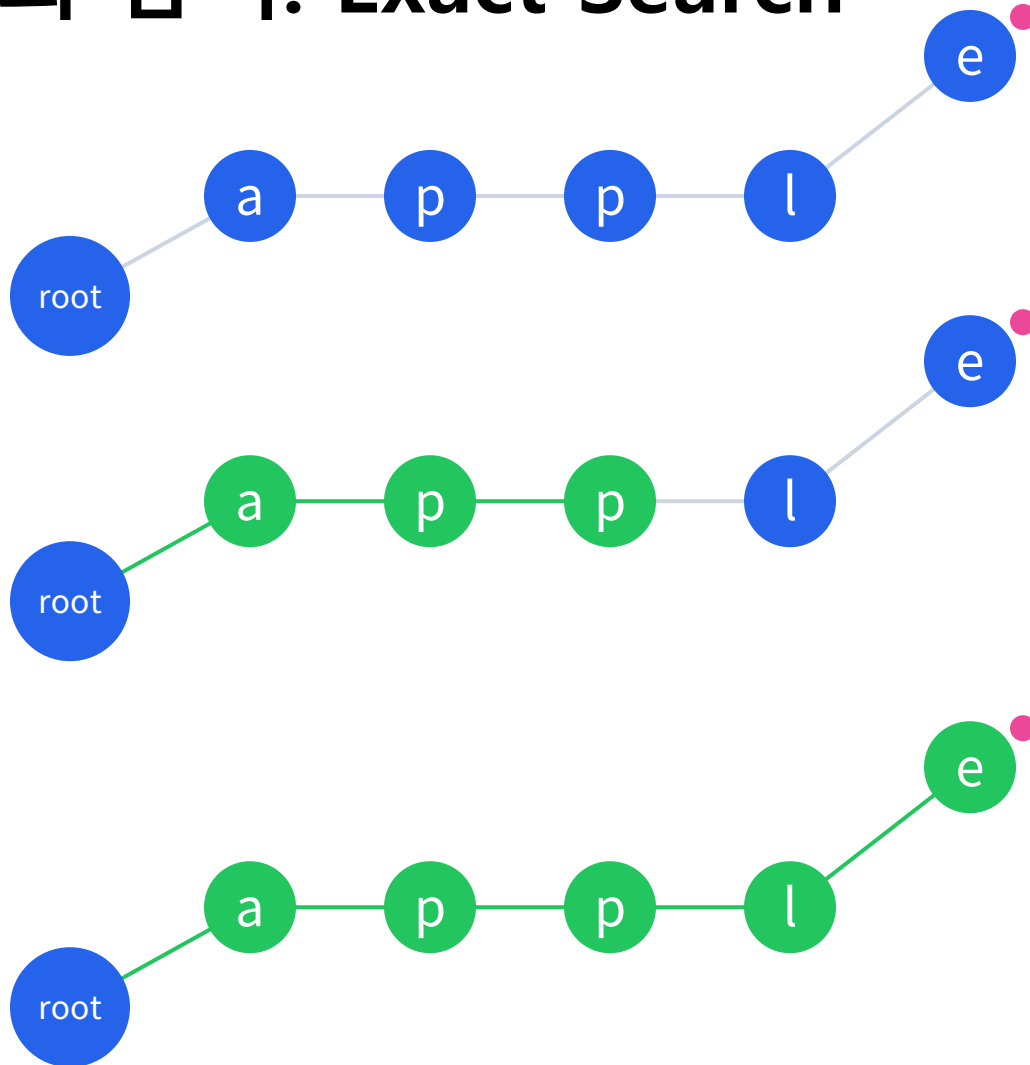


End Marker



저장된 단어가 끝나는 지점에 별도 표시

Trie의 탐색: Exact Search

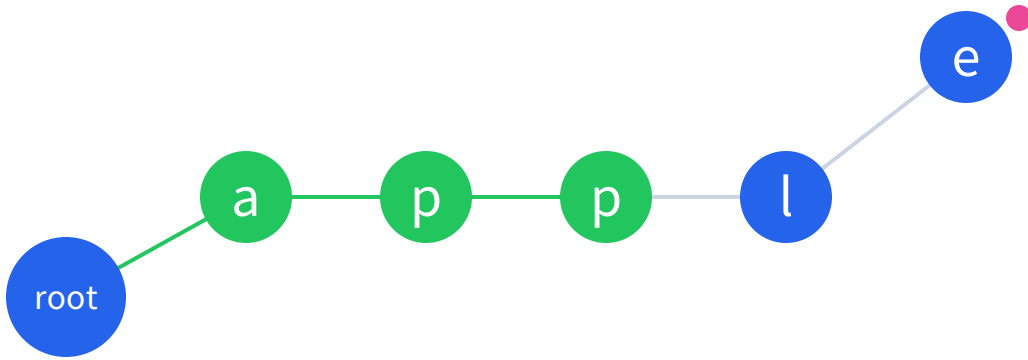


search("apple")

Exact Search는 모든 문자 경로와
End Marker를 확인함

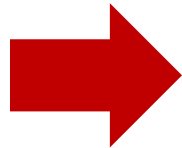
➡ "app"이 저장되어 있나?

Trie의 탐색: Prefix Search



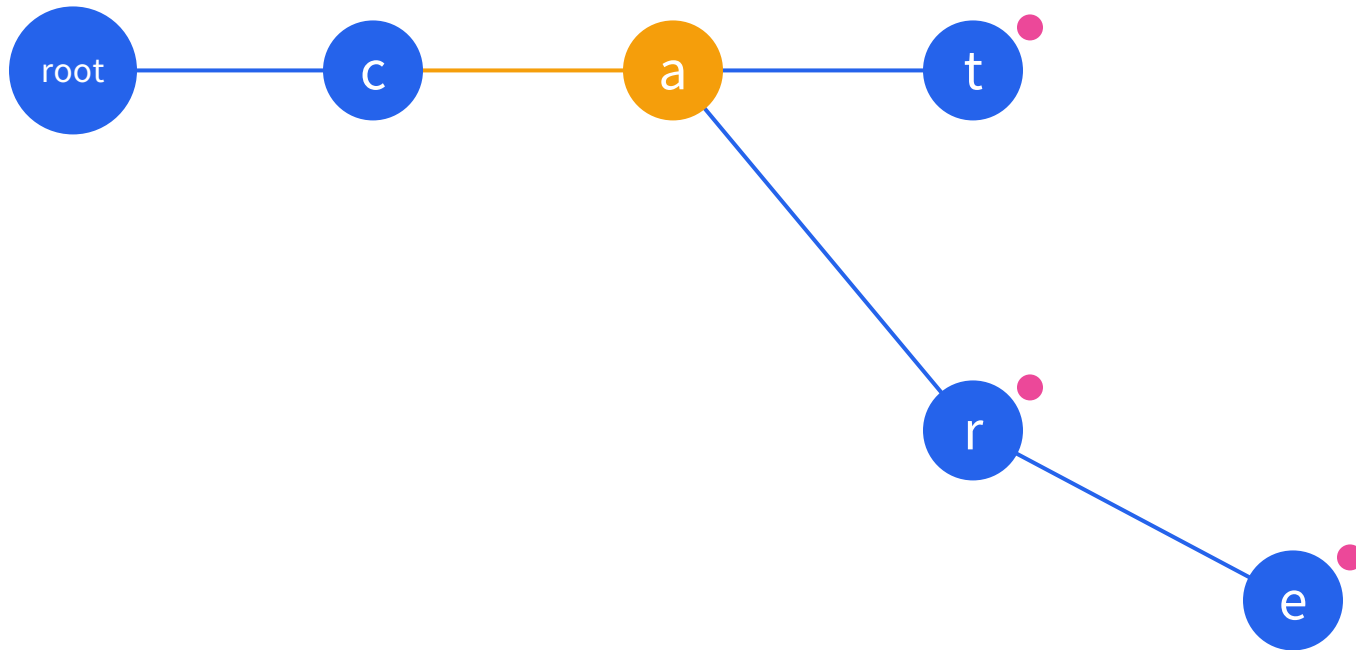
Query	필요한 조건	apple만 저장한 경우
Exact(app)	Path + END	False
Prefix(app)	Path	True

Prefix Search에서는 End Marker가
True가 아니어도 됨
"app"으로 시작하는
단어가 있는가?



01 Trie

Trie의 탐색: 원하는 값이 없을 경우



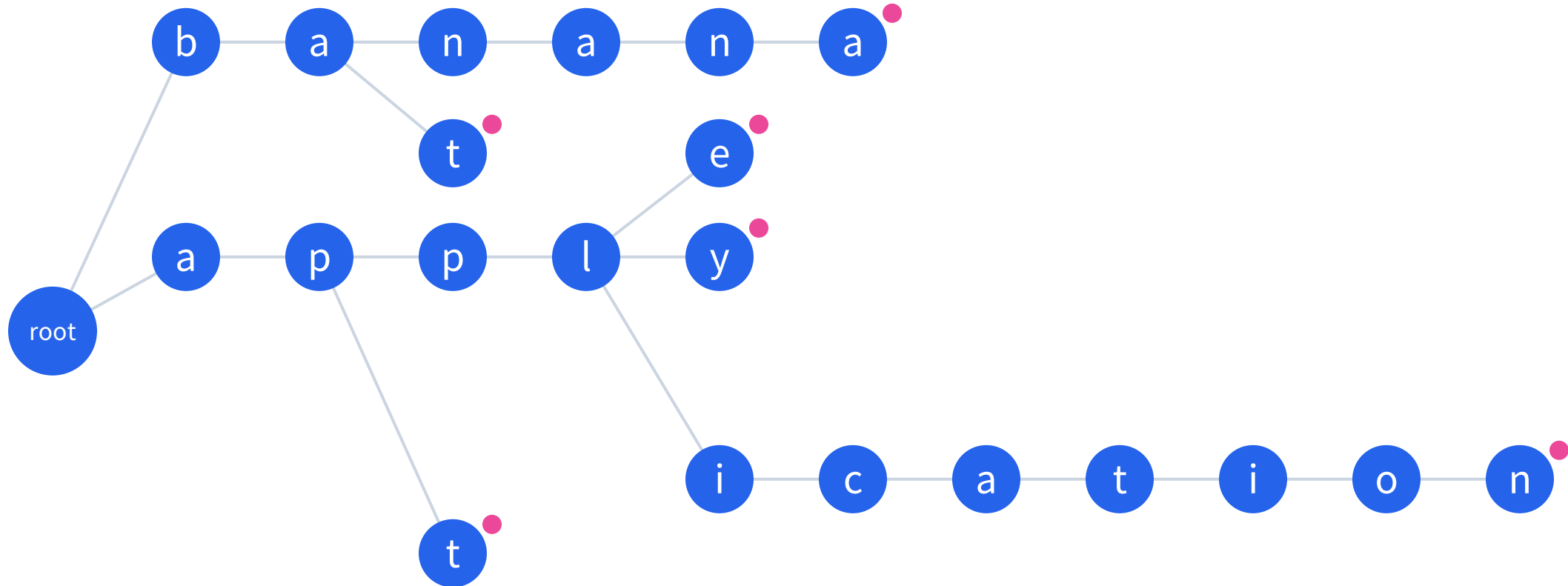
c ✓ a ✓ n 없음

즉시 False

(전체 Trie를
다 뒤지지 않아도 됨)

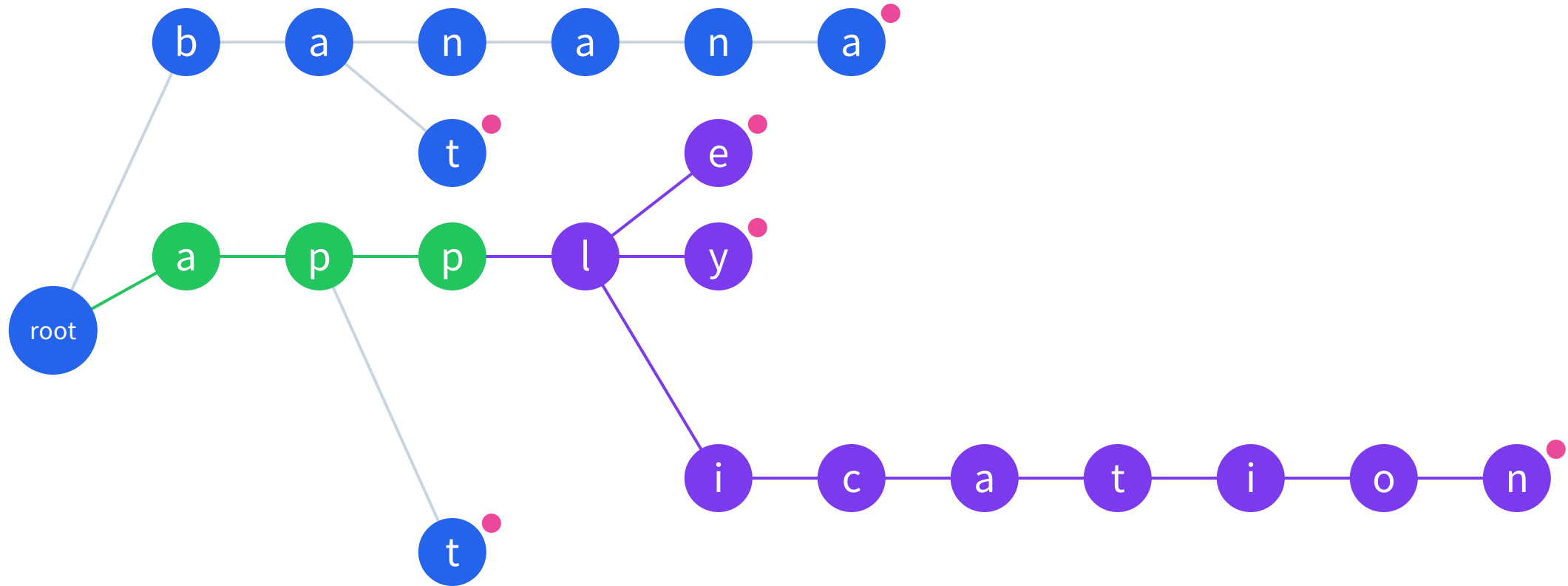
01 Trie

Trie를 이용한 자동완성



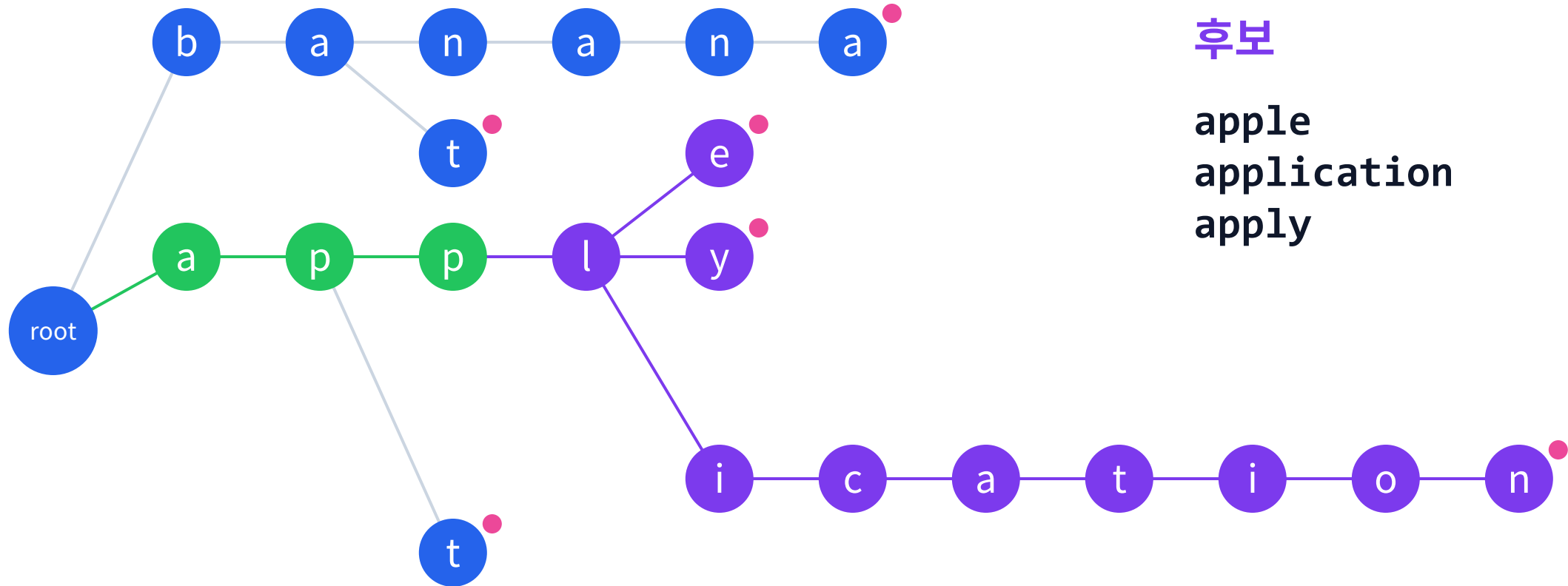
Trie에 저장된 문자열들

Trie를 이용한 자동완성



사용자가 "app" 입력

Trie를 이용한 자동완성



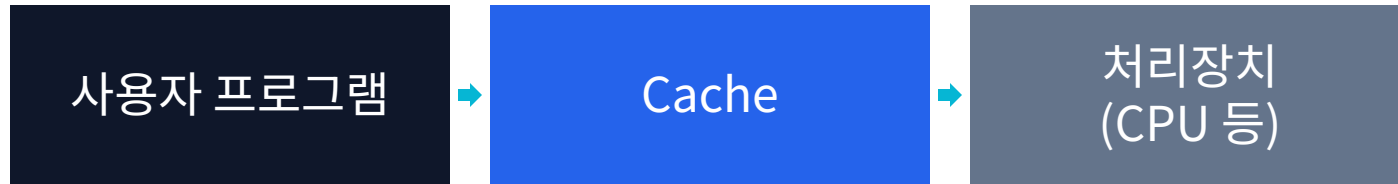
Subtree에서 후보 추출

02

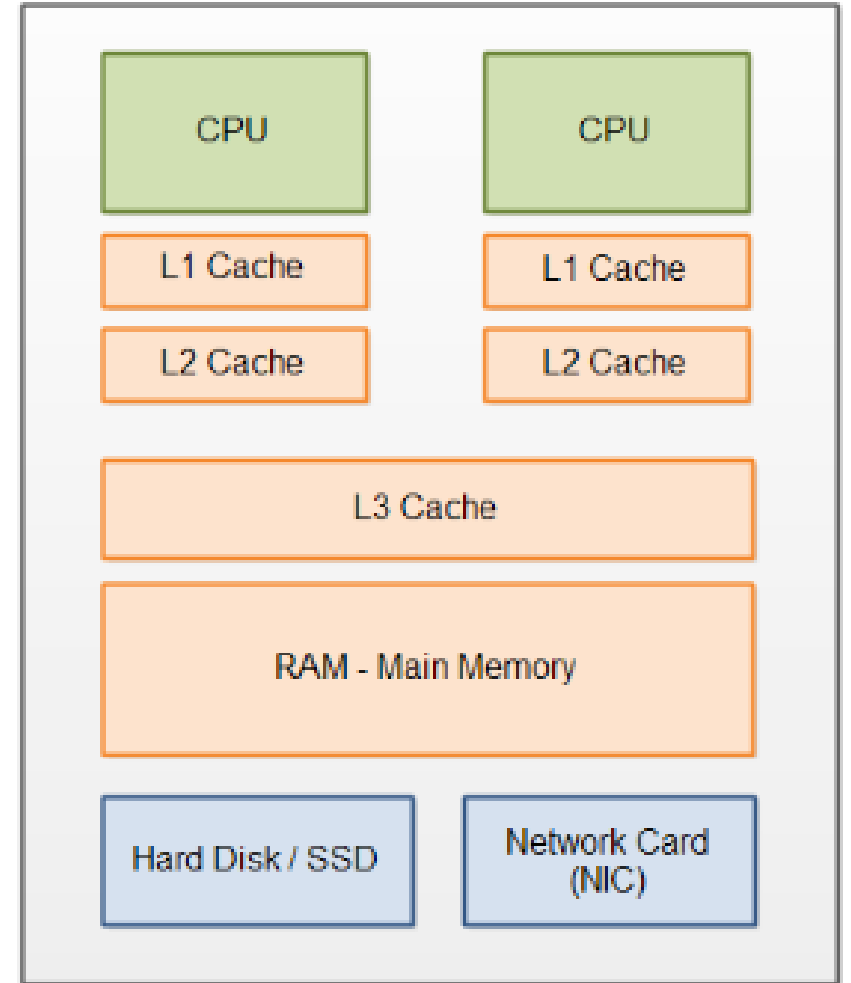
캐시

LRU Cache

캐시



빠르게 쓸 일부 데이터를 가까운 공간에 저장



CPU 안의 캐시

Cache Hit

LRU
Least Recently Used

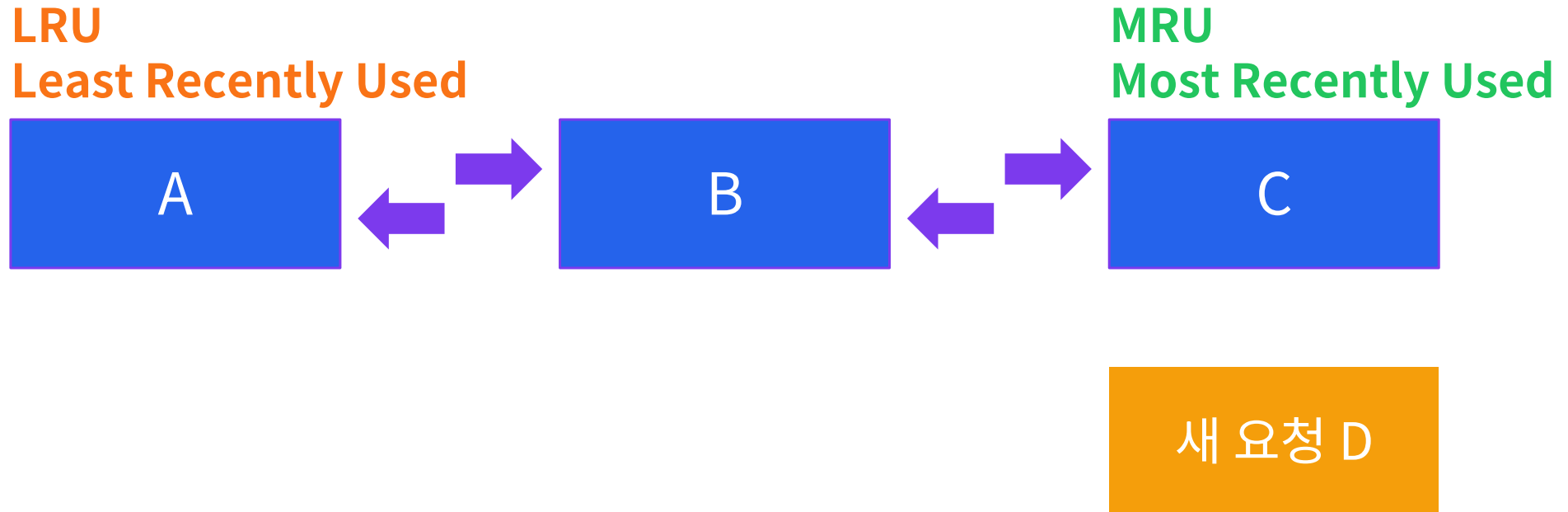


MRU
Most Recently Used



요청 데이터가 캐시에 있음: 바로 사용

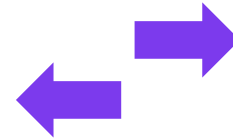
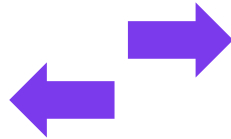
Cache Hit



요청 데이터가 캐시에 없음: 원본에서 데이터를 읽어옴

Cache Capacity

LRU
Least Recently Used



MRU
Most Recently Used



만약 Cache Capacity가 3인데 새로운 요청이 발생하면?

Cache Capacity

진 행 순 서



FIFO: 선입 선출



LRU: 마지막 사용 시점



A, B, C 적재

Cache Capacity



FIFO: 선입 선출



LRU: 마지막 사용 시점



FIFO는 적재 순서 유지
LRU는 A를 MRU인 오른쪽으로 이동

Cache Capacity

진행 순서



FIFO: 선입 선출



LRU: 마지막 사용 시점



FIFO는 A 제거
LRU는 B 제거

LRU의 핵심 연산

Key Lookup

존재 확인과 Value 얻기

Recency Update

사용한 Node를 MRU 쪽으로 이동

Eviction / Insert

왼쪽 LRU 제거와 새 Entry 추가

LRU의 핵심 연산

단계	사용 구조	교육용 비용
Key Lookup	Hash Table	평균 $O(1)$
Detach / MRU 연결	Doubly Linked List	웬만하면 $O(1)$
LRU 선택 / 삭제	List 끝 + Hash 삭제	평균 $O(1)$

get()

Lookup + Detatch + MRU 연결

put()

Lookup

+ Detatch + MRU 연결 (기존 Key일 때)

+ 새 Node → MRU 연결 (새 Key일 때)

+ LRU 선택 / 삭제 (용량 초과 시)

LRU의 핵심 연산

GET

PUT

현재 동작

초기 상태

B를 조회합니다.

1 / 6

LRU

MRU

A ↔ B ↔ C

연산

get(B)

←

다음 →

자료구조 결합

Dijkstra

Graph + Heap + dist

LRU

Hash Table + Doubly Linked List

Trie 구현

Tree 구조 + Hash Map Children

자료구조는 하나만 쓸 필요가 없음

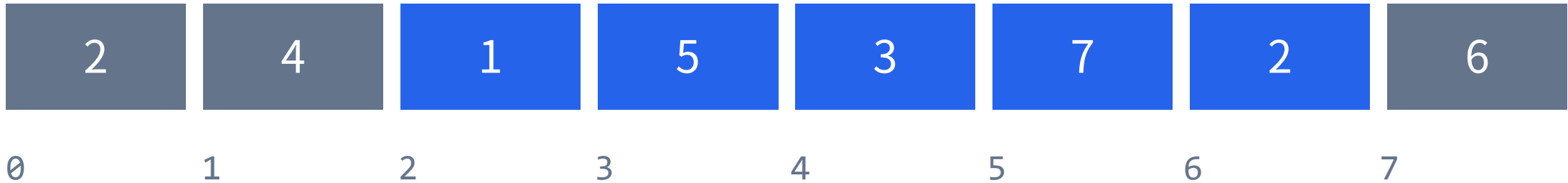
03

Range Query

Range Query

03 Range Query

Range Sum (구간합)



2번 인덱스부터 6번 인덱스까지의 합을 구하려면? (`sum(list[2:7])`)
와 같은 구간합 문제가 100만개 들어온다면? (Query가 100만개라면?)



계속 같은 연산을 반복하게 되는 문제

03 Range Query

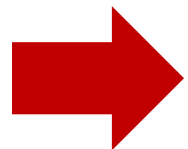
Prefix Sum



$A[2]: 1 \rightarrow 10$, 변화량 +9



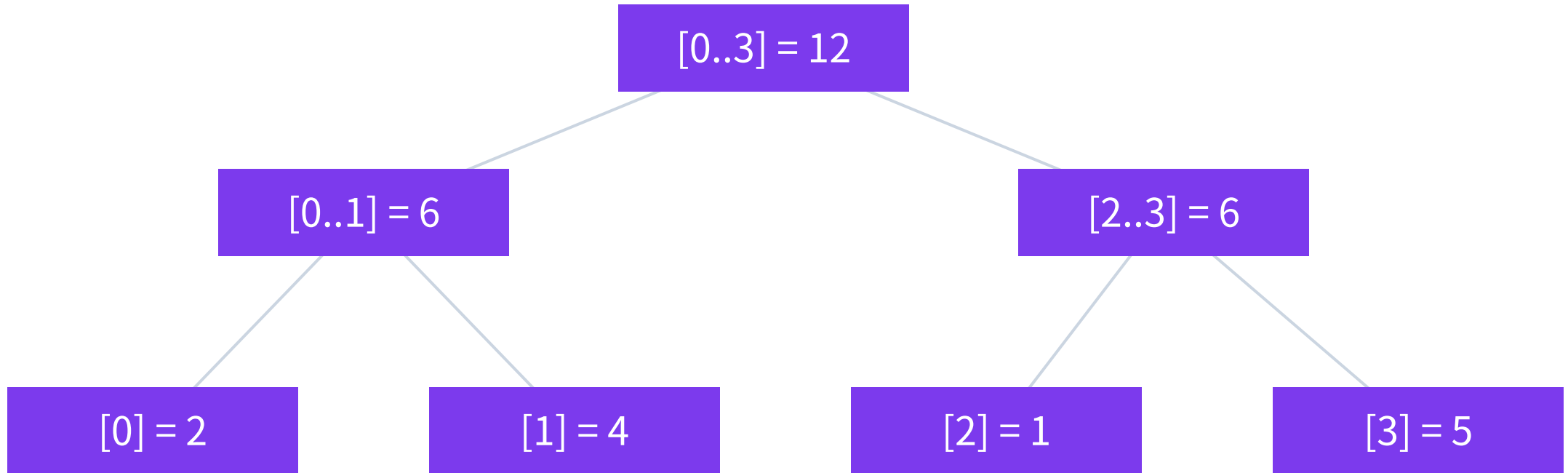
원본 배열의 값이 달라지면 이후의 Prefix도 전부 고쳐야 함



그런데 Query도 많고 Update도 많으면?

03 Range Query

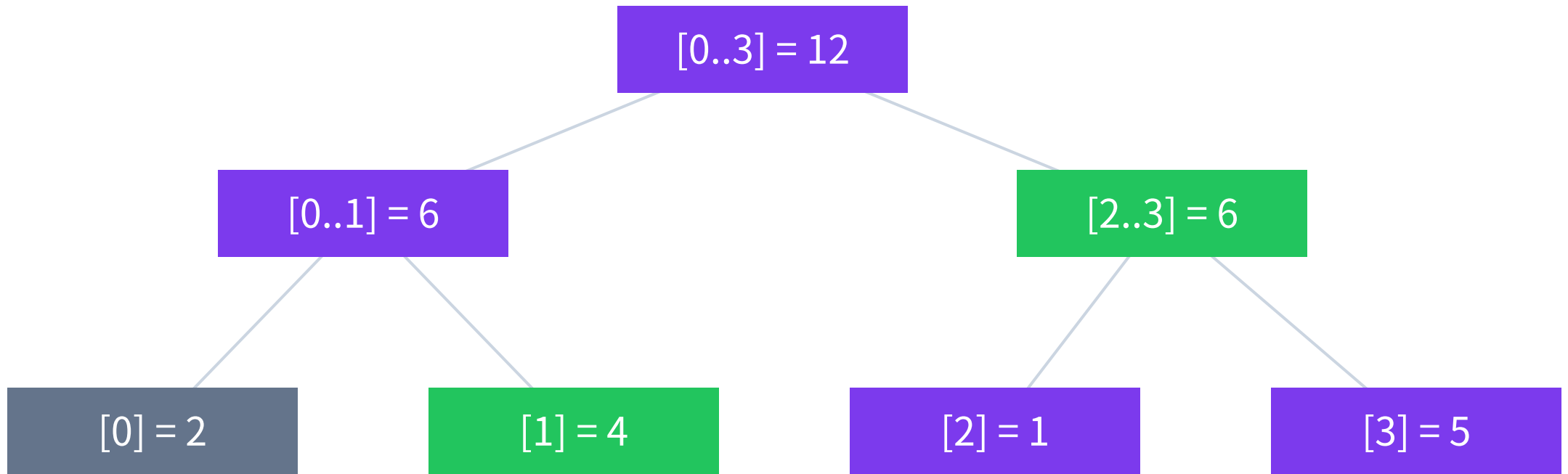
Segment Tree



구간합을 구할 때, 구간을 쪼갬 후
각 구간의 합을 트리에 저장

03 Range Query

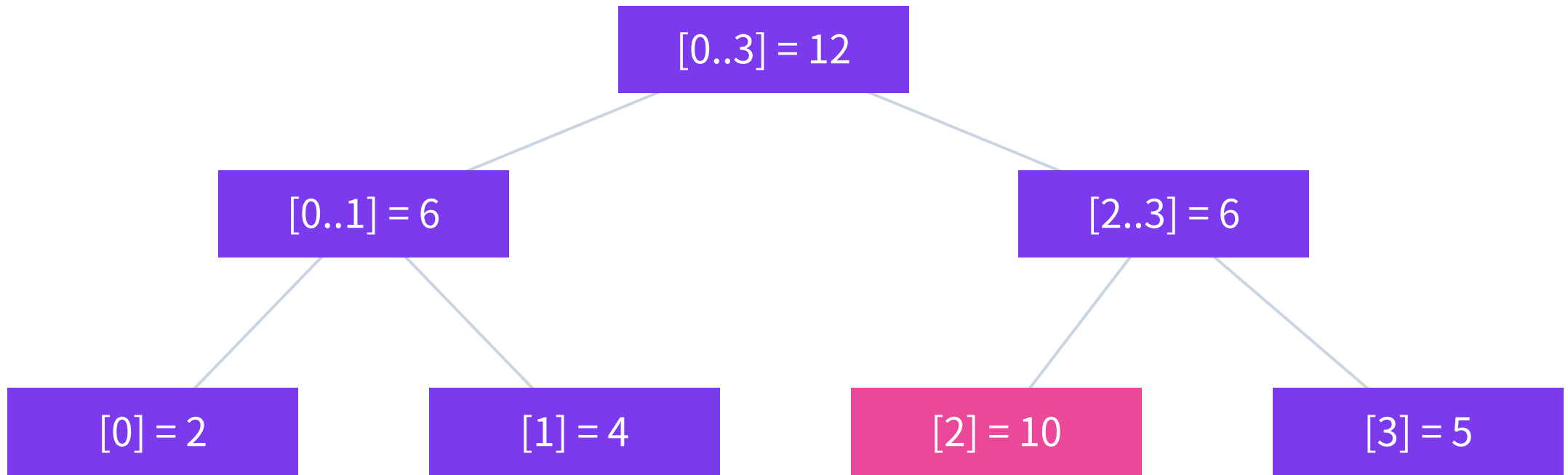
Segment Tree Query: $\text{sum}(\text{Array}[1:4])$



$\text{sum}(\text{Array}[1:4]) = A[1] + A[2] + A[3]$
해당하는 범위를 찾을 때 까지 Root 에서부터 탐색

03 Range Query

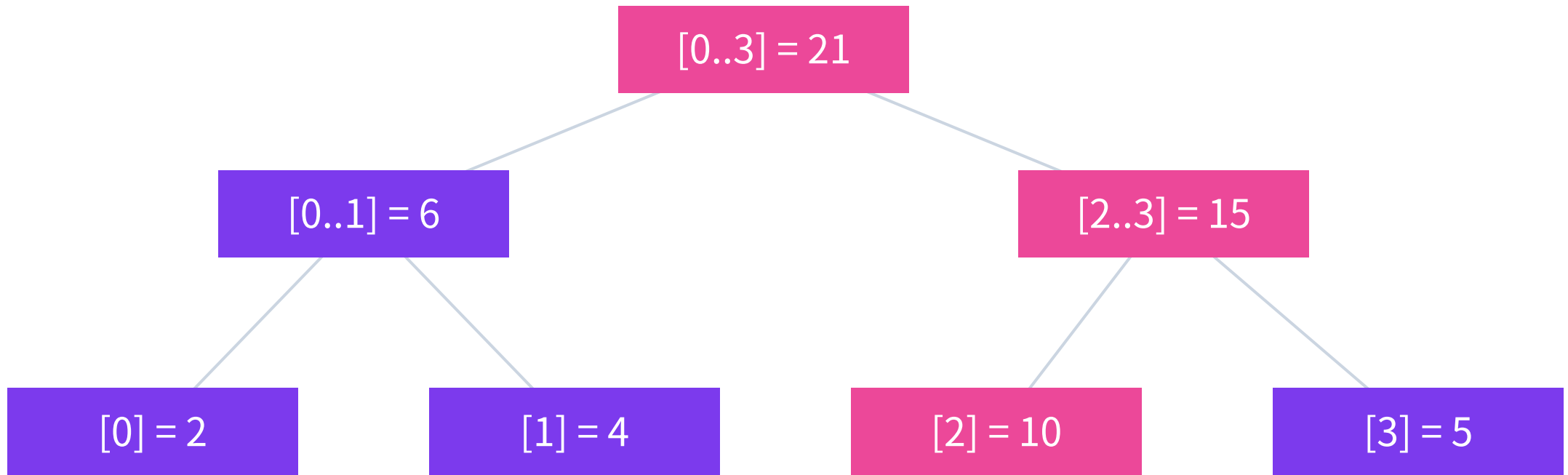
Segment Tree Update: $A[2] = 10$



리프노드 그냥 업데이트

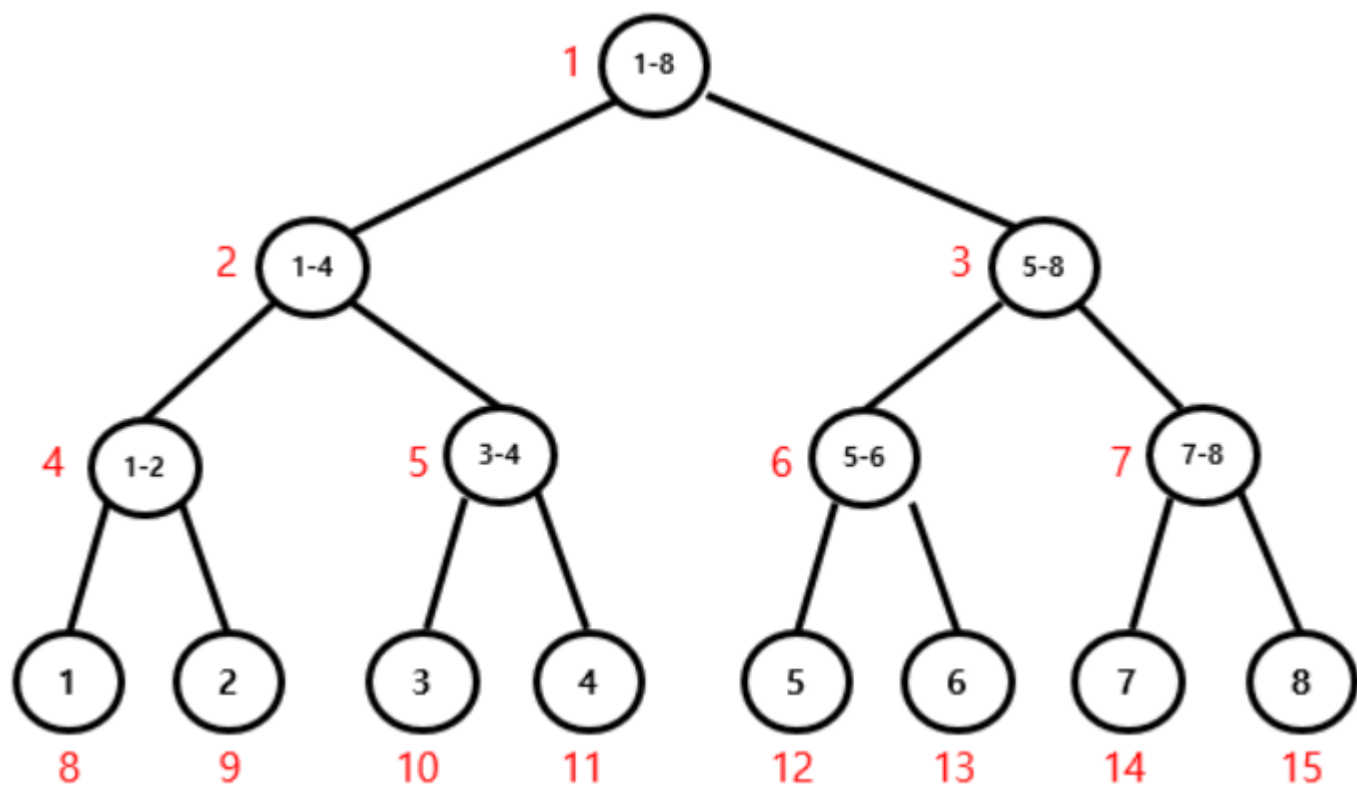
03 Range Query

Segment Tree Update: $A[2] = 10$



이후 영향을 받은 부모들만 다시 고침

Fenwick Tree



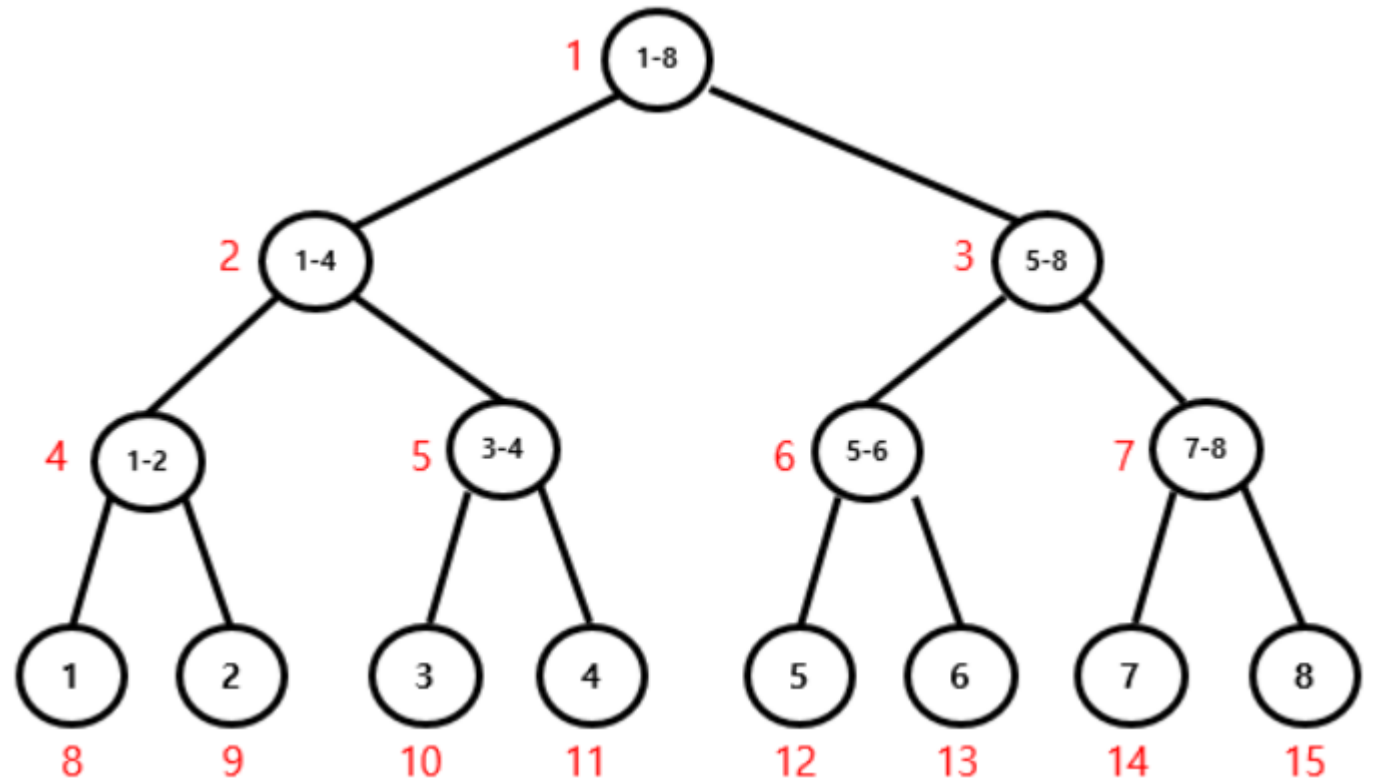
배열의 Index

	1	2	3	4	5	6	7	8
1	1	3	3	5	5	7	7	
2	1 - 2			6	5 - 6			
4	1 - 4							
8	1 - 8							

03 Range Query

Fenwick Tree

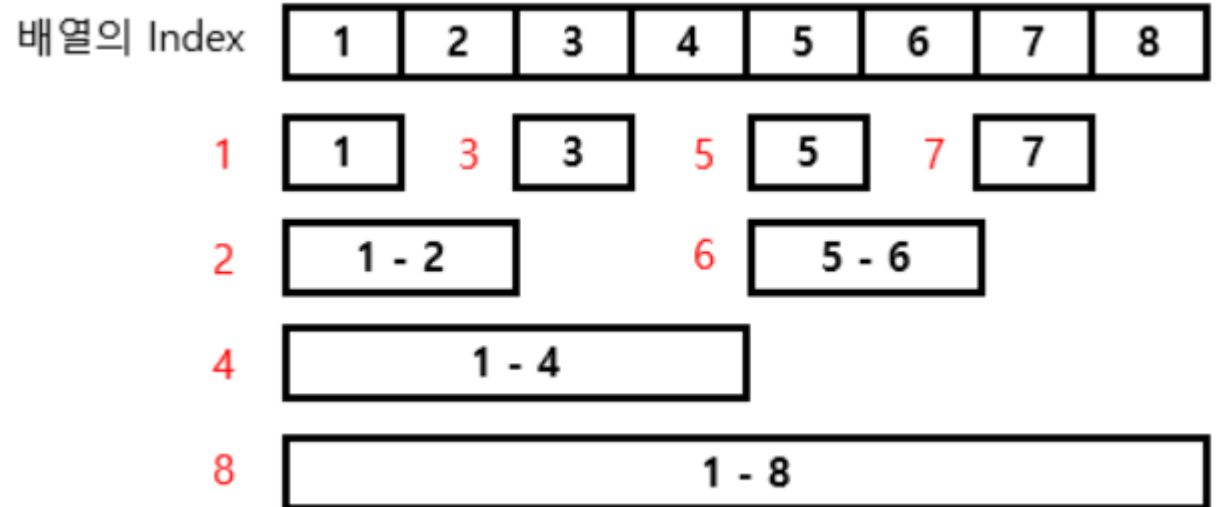
- 각 동그라미는 노드를 의미
- 노드 안의 숫자는 배열의 인덱스 번호
- 빨간 숫자들은 세그먼트 트리의 인덱스 번호
- 구간합에 중점을 둔 세그먼트 트리와 달리
펜윅트리는 누적합에 중점을 두도록 설계됨



03 Range Query

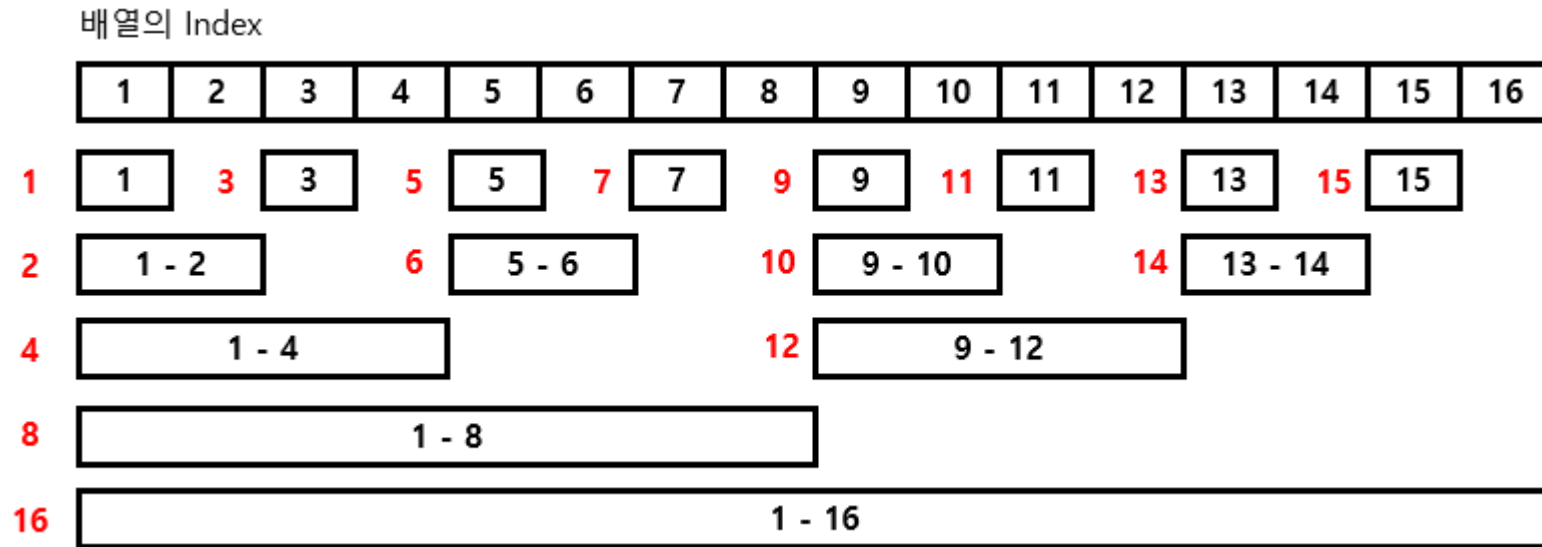
Fenwick Tree

- 크기가 8인 배열에 대한 펜윅트리
- “배열의 Index” 를 제외한 사각형들은 펜윅트리에 들어가 있는 값들을 의미함
- 빨간 숫자는 펜윅트리 인덱스 번호를 의미
- 홀수 인덱스들은 배열의 값을 그대로 저장
- 비트를 이용해 연산



03 Range Query

Fenwick Tree



배열의 길이가 16일때의 펜윅트리 생성과정

03 Range Query

Fenwick Tree

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----

0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111	10000
------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	-------

각 인덱스들을 모두 이진수로 변환

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----

0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111	10000
------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	-------

1	2	1	4	1	2	1	8	1	2	1	4	1	2	1	16
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	----

각 이진수마다 가장 오른쪽의 1이 의미하는 숫자를 나타냄

03 Range Query

Fenwick Tree

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----

0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111	10000
------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	-------

1	2	1	4	1	2	1	8	1	2	1	4	1	2	1	16
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	----

배열의 Index

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	1	3	3	5	5	7	7	9	9	11	11	13	13	15	15	
2	1 - 2			6	5 - 6			10	9 - 10			14	13 - 14			
4	1 - 4							12	9 - 12							
8	1 - 8															
16	1 - 16															

03 Range Query

Fenwick Tree

작업	시간복잡도	공간복잡도
Query	$O(\log N)$	$O(N)$
Update	$O(\log N)$	$O(N)$
Range Query	$O(\log N)$	$O(N)$

03 Range Query

Prefix Sum / Segment Tree / Fenwick Tree

구조	Query	Update	Range Query	특징
Prefix Sum	$O(1)$	$O(N)$	$O(1)$	값이 거의 바뀌지 않는 데이터에 적합
Segment Tree	$O(\log N)$	$O(\log N)$	$O(\log N)$	구간합, 최솟값, 최댓값 등 다양한 연산 가능
Fenwick Tree	$O(\log N)$	$O(\log N)$	$O(\log N)$	누적합 + 값 변경이 필요할 때 세그먼트 트리보다 구현이 간단